

開発環境を整える 〜プログラミングの「キッチン」を準備しよう〜

はじめに

「プログラミングを始めたいけど、環境構築でつまずいて、コードを書く前に心が折れそう…」そんな経験はありませんか？

Node.js（ノードジェイエス）やnpm（エヌピーエム）、Docker（ドッカー）やデータベース（データを保存する場所）など、新しい言葉がたくさん出てきて、何から手をつければいいのか分からなくなるのは当然です。コマンド（コンピュータへの指示）を実行するのも、何が起こるか分からなくて怖いかもしれません。そして、エラーが出た時に、どこを疑えばいいのか見当もつかない…。

でも、大丈夫！この章が終わる頃には、あなたのPCの中に開発専用の「キッチン」が完成し、必要な道具と材料がすべて揃って、スムーズにアプリを開発できる状態になります。

解決策

プログラミングの開発環境を、料理をするための「キッチン」に例えてみましょう。

- **Node.js（ノードジェイエス：JavaScriptをPC上で動かす実行環境）**：「コンロ（火元）」です。JavaScriptというプログラミング言語を動かすための熱源となります。
- **npm（エヌピーエム：必要な道具を集めて実行もできる管理ツール）**：「キッチンの受付/注文票」です。必要な調理器具（ライブラリ）を注文

し、レシピ（スクリプト）を実行するための窓口です。

- **npm scripts（エヌピーエム・スクリプト：よく使うコマンドに名前を付けたショートカット）**：「ワンタッチ調理ボタン」です。「炒める」「煮る」のように、よく使う処理をボタン一つで実行できます。
- **Docker（ドッカー：アプリやDBを箱詰めして同じ環境で動かす仕組み）**：「食材キット（箱）」です。誰のPC（キッチン）でも、同じ材料と手順で料理（アプリ）を作れるようにします。
- **Docker Compose（ドッカー・コンポーズ：複数の箱をまとめて起動する指揮係）**：「献立表」です。データベース（冷蔵庫）とバックエンド（調理台）をセットで立ち上げる手順を記述します。
- **Database / PostgreSQL（ポストグレスキューエル：データを保存する倉庫）**：「冷蔵庫」です。食材（データ）を安全に保管します。

まずは「動く状態」を最優先にしましょう。完璧な理解は後からでOKです。もし失敗しても、コマンドは「手順書」通りに実行すれば、いつでも元の状態に戻れます。大切なのは、どこで詰まったかを切り分けることです。

- npmが動かない？ → Node.js/npmの問題
- DBが動かない？ → Docker/Composeの問題
- アプリが起動しない？ → scripts/環境変数（プログラムの設定値）の問題

【スクリーンショット】Node.js、npm、Docker、それぞれのロゴ

実際のコード

3.1 “ワンタッチ調理ボタン”を確認する（npm scripts）

`package.json` は、あなたのアプリのレシピカードです。その中にある

`scripts` は、よく使うコマンドを簡単に実行できるようにするための「ワンタッチ調理ボタン」のリストです。

```
"dev": "next dev",
"build": "prisma generate && prisma db push --accept-
"build:ci": "prisma generate && next build",
"vercel-build": "prisma generate && next build",
"start": "next start",
"lint": "biome check .",
"lint:fix": "biome check --write .",
"format": "biome format --write .",
"db:push": "prisma db push",
"db:studio": "prisma studio",
"db:generate": "prisma generate"
```

- 1行目の `"dev": "next dev"` は、「開発」ボタンを押すと `next dev` というコマンドが実行されることを意味します。`next dev` は、Next.js（ネクストジェイエス：ReactをベースにしたWebフレームワーク）の開発サーバーを起動するコマンドです。
- 2行目の `"build": "prisma generate && prisma db push --accept-data-loss && next build"` は、「ビルド」ボタンを押すと、本番環境向けにアプリを構築する一連のコマンドが実行されます。`prisma generate` はPrisma（プリズマ：データベースを扱いやすくするツール）のコードを生成し、`prisma db push` はデータベースのスキーマ（構造）を更新し、`next build` はNext.jsのアプリをビルドします。
- 5行目の `"start": "next start"` は、「スタート」ボタンを押すと、ビルドされたアプリを起動します。
- 6行目の `"lint": "biome check ."` は、「リント」ボタンを押すと、コードのスタイル（書き方）をチェックします。Biome（バイオーム：コードの整形ツール）を使って、コードが読みやすく、一貫性があるかを確認します。

これらのボタンは、ターミナル（コマンドを入力する画面）で `npm run dev` のように `npm run` の後にボタンの名前を入力するだけで実行できます。

`build` は少し重い作業なので、まずは `dev` で起動を確認し、次に `build` を試すのがおすすめです。

【スクリーンショット】 `package.json` の `scripts` 部分をハイライト

3.2 “冷蔵庫 (DB)” を箱で用意する (PostgreSQL on Docker)

`docker-compose.yml` は、あなたのキッチンの献立表です。まずは冷蔵庫 (データベース) を用意する部分を見てみましょう。

```
services:
  db:
    image: postgres:16-alpine
    container_name: taskapp-postgres
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: taskapp
    ports:
      - "${_DOCKER_COMPOSE_HOST_PORT_DB:-5432}:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5
```

- `services:` は、この献立表で管理するサービス (部品) を定義する場所です。
- `db:` は、データベースという名前のサービスを定義します。これが冷蔵庫の名前になります。

- `image: postgres:16-alpine` は、冷蔵庫の型番を指定します。ここでは、PostgreSQL（ポストグレスキューエル：データを保存する大きな台帳）のバージョン16の軽量版を使います。
- `container_name: taskapp-postgres` は、Dockerコンテナ（コンテナ：アプリを動かすための隔離された環境）の名前を指定します。
- `environment:` は、冷蔵庫の初期設定です。 `POSTGRES_USER`、`POSTGRES_PASSWORD`、`POSTGRES_DB` は、それぞれデータベースのユーザー名、パスワード、データベース名を指定します。これらは環境変数（プログラムの設定値）として設定されます。
- `ports:` は、家のドア番号（PC側）と冷蔵庫の差し込み口（コンテナ側）を繋げます。 `5432` はPostgreSQLの標準的なポート（ポート：アプリが外界と通信するための「出入口」）番号です。
`$_DOCKER_COMPOSE_HOST_PORT_DB:-5432` は、環境変数 `_DOCKER_COMPOSE_HOST_PORT_DB` が設定されていなければ、デフォルトで `5432` を使うという意味です。
- `volumes:` は、冷蔵庫の中身（データ）をPCに保存するための場所を指定します。 `postgres-data:/var/lib/postgresql/data` は、コンテナ内の `/var/lib/postgresql/data` というディレクトリ（フォルダ）を、PC上の `postgres-data` という名前のボリューム（データの保存場所）に接続します。これにより、コンテナを削除してもデータが失われません。
- `healthcheck:` は、冷蔵庫が正常に動いているか定期的に確認する機能です。 `pg_isready -U user` というコマンドを実行して、データベースが使える状態かどうかをチェックします。intervalはチェックの間隔（10秒）、timeoutはタイムアウト（5秒）、retriesはリトライ回数（5回）を意味します。

データベースは、アプリとは別の箱（コンテナ）で動かすことで、互いに影響を与えずに開発を進めることができます。

【スクリーンショット】 docker-compose.ymlのdbサービスをハイライト

3.3 “献立表”でバックエンドも一緒に立ち上げる（multi-service）

次に、調理台（バックエンド）も一緒に立ち上げるための設定を見てみましょう。

```
backend:
  build:
    context: .
    dockerfile: .docker/backend/Dockerfile
  container_name: taskapp-backend
  ports:
    - "${_DOCKER_COMPOSE_HOST_PORT_BACKEND:-3000}:3000"
    - "${_DOCKER_COMPOSE_HOST_PORT_BACKEND_DEBUG:-9229}
  env_file:
    - .env
  environment:
    - DATABASE_URL=postgresql://user:password@db:5432/t
  volumes:
    - ./:/app:delegated
    - backend-nodemodules-volume:/app/node_modules/
  depends_on:
    db:
      condition: service_healthy
```

- `backend:` は、バックエンドという名前のサービスを定義します。これが調理台の名前になります。
- `build:` は、この箱（コンテナ）をどのように作るかを指定します。
`context: .` は、Dockerfile（ドッカーファイル：コンテナの設計図）がある場所（現在のディレクトリ）を指定します。
`dockerfile: .docker/backend/Dockerfile` は、Dockerfileのパスを指定します。
`build` は、既製品（image）がない場合に、自分でコンテナを組み立てる必要がある時に使います。

- `ports:` は、外部からのアクセスを受け付けるための入口です。 `3000` 番ポートは、Webブラウザからアプリにアクセスするためのポートです。
`9229` 番ポートは、デバッグ（プログラムの誤りを見つける作業）用のポートです。
- `env_file: - .env` は、環境変数を `.env` ファイルから読み込むように指定します。
- `environment:` は、環境変数を直接指定します。 `DATABASE_URL` は、データベースへの接続情報です。
- `volumes:` は、ソースコードをコンテナに反映させるための設定です。 `./:/app:delegated` は、現在のディレクトリ（`.`）をコンテナ内の `/app` ディレクトリにマウント（接続）します。 `:delegated` は、PC側の変更がコンテナにすぐに反映されるようにするためのオプションです。
`backend-nodemodules-volume:/app/node_modules/` は、Node.jsのモジュール（部品）を保存するためのボリュームです。
- `depends_on: db` は、バックエンドを起動する前に、データベース（冷蔵庫）が先に起動している必要があることを指定します。これは「順番待ちのルール」のようなものです。 `condition: service_healthy` は、データベースが正常に動作している状態（healthcheckが成功している状態）でのみ、バックエンドを起動することを意味します。

【スクリーンショット】 docker-compose.ymlのbackendサービス部分をハイライト

動かしてみよう

a) 前提チェック（Prerequisites Check）

まず、あなたのPCにNode.js、npm、Dockerがインストールされているか確認しましょう。

1. **Node.jsの確認:** ターミナルで以下のコマンドを実行してください。

```
$ node -v  
v24.11.1
```

`v24.11.1`のようにバージョンが表示されればOKです。もし `v20.xx.xx` のように古いバージョンが表示された場合は、Node.jsのバージョンをアップデート（更新）する必要があります。

2. **npmの確認**: ターミナルで以下のコマンドを実行してください。

```
$ npm -v  
10.2.4
```

`10.2.4`のようにバージョンが表示されればOKです。

3. **Dockerの確認**: ターミナルで以下のコマンドを実行してください。

```
$ docker --version  
Docker version 26.1.0, build 97df0a1
```

`Docker version 26.1.0, build 97df0a1`のようにバージョンが表示されればOKです。もしDockerがインストールされていない場合は、Docker Desktopをインストールしてください。

【スクリーンショット】ターミナルで `node -v`、`npm -v`、`docker --version` を実行した結果が出ている画面

b) Step-by-step verification

1. **依存関係を揃える**: プロジェクトのルートディレクトリ（`package.json`がある場所）で、以下のコマンドを実行してください（初回のみ）。


```
$ npm install
```

このコマンドは、`package.json` に書かれている必要なライブラリをすべてインストールします。

2. **DB（冷蔵庫）を起動**: ターミナルで以下のコマンドを実行してください。

```
$ docker compose up -d db
```

`-d` オプションは、バックグラウンドでコンテナを起動することを意味します。

3. **バックエンドも起動**: ターミナルで以下のコマンドを実行してください。

```
$ docker compose up -d
```

このコマンドは、`docker-compose.yml` に書かれているすべてのサービス（DBとバックエンド）を起動します。

4. **アプリを起動**: ターミナルで以下のコマンドを実行してください。

```
$ npm run dev
```

このコマンドは、`package.json` に定義されている `dev` スクリプトを実行し、開発サーバーを起動します。

5. **ブラウザで確認**: Webブラウザで `http://localhost:3000` を開いてください。Next.jsのウェルカムページが表示されれば成功です。

【スクリーンショット】 `docker compose up` 実行中ログで、db が「healthy」と表示され、backend が「localhost:3000 ready」となっている部

分

c) Visual confirmation checklist

- 【スクリーンショット】 Docker Desktopで「taskapp-postgres」と「taskapp-backend」の2つのコンテナが「Running」状態になっていることを確認
- 【スクリーンショット】 ターミナルで `node -v` の実行結果が出ている画面
- 【スクリーンショット】 `docker compose up` 実行中ログで、db が「healthy」と表示され、backend が「localhost:3000 ready」となっている画面
- 【スクリーンショット】 ブラウザで `http://localhost:3000` を開いた結果、Next.jsのウェルカムページが表示されている画面

d) Troubleshooting tips

- “**port already in use**”: 既に同じポート（出入口）を使っているアプリがあります。ターミナルで `lsof -i :3000`（3000はポート番号の例）を実行して、どのプロセス（アプリ）がポートを使っているか確認し、不要なプロセスを終了してください。
- “**docker not running**”: Docker Desktopが起動しているか確認してください。起動していない場合は、Docker Desktopを起動してください。
- “**cannot connect to db**”: DBコンテナが起動しているか、環境変数（特に `DATABASE_URL`）が正しく設定されているか確認してください。

まとめ

開発環境はキッチンです。npm scriptsで作業をボタン化し、DockerでDBという冷蔵庫を用意すれば、迷わず調理（開発）を始められます。